# Javadoc

Carsten Gips (HSBI)

**TL;DR**
Mit Javadoc kann aus speziell markierten Block-Kommentaren eine externe Dokumentation im HTML-Format erzeugt werden. Die Block-Kommentare, auf die das im JDK enthaltene Programm `javadoc` reagiert, beginnen mit `/**` (also einem zusätzlichen Stern, der für den Java-Compiler nur das erste Kommentarzeichen ist).
Die erste Zeile eines Javadoc-Kommentars ist eine "Zusammenfassung" und an fast allen Stellen der generierten Doku sichtbar. Diese Summary sollte kurz gehalten werden und eine Idee vermitteln, was die Klasse oder die Methode oder das Attribut macht.
Für die Dokumentation von Parametern, Rückgabetypen, Exceptions und veralteten Elementen existieren spezielle Annotationen: `@param`, `@return`, `@throws` und `@deprecated`.
Als Faustregel gilt: Es werden **alle** **`public`** und **`protected`** Elemente (Klassen, Methoden, Attribute) mit Javadoc kommentiert. Alle nicht-öffentlichen Elemente bekommen normale Java-Kommentare (Zeilen- oder Blockkommentare).

**Videos**

- VL Javadoc

**Unterlagen**

- PDF-Version

---

# 1 Dokumentation mit Javadoc

```
/**
 * Beschreibung Beschreibung (Summary).
 *
 * <p>Hier kommt dann ein laengerer Text, der die Dinge
 * bei Bedarf etwas ausfuehrlicher erklaert.
 */
public void wuppie() {}
```

Javadoc-Kommentare sind (aus Java-Sicht) normale Block-Kommentare, wobei der Beginn mit `/**` eingeleitet wird. Dieser Beginn ist für das Tool `javadoc` (Bestandteil des JDK, genau wie `java` und `javac`) das Signal, dass hier ein Kommentar anfängt, den das Tool in eine HTML-Dokumentation übersetzen soll.

Typischerweise wird am Anfang jeder Kommentarzeile ein `*` eingefügt; dieser wird von Javadoc ignoriert.

Sie können neben normalem Text und speziellen Annotationen auch HTML-Elemente wie `<p>` und `<code>` oder `<ul>` nutzen.

Mit `javadoc *.java` können Sie in der Konsole aus den Java-Dateien die Dokumentation generieren lassen. Oder Sie geben das in Ihrer IDE in Auftrag … (die dann diesen Aufruf gern für Sie tätigt).

# 2 Standard-Aufbau

```
/**
 * Beschreibung Beschreibung (Summary).
 *
 * <p> Hier kommt dann ein laengerer Text, der die Dinge
 * bei Bedarf etwas ausfuehrlicher erklaert.
 *
 * @param   date  Tag, Wert zw. 1 .. 31
 * @return  Anzahl der Sekunden seit 1.1.1970
 * @throws  NumberFormatException
 * @deprecated As of JDK version 1.1
 */
public int setDate(int date) {
    setField(Calendar.DATE, date);
}
```

- Erste Zeile bei Methoden/Attributen geht in die generierte "Summary" in der Übersicht, der Rest in die "Details"
  - Die "Summary" sollte kein kompletter Satz sein, wird aber wie ein Satz geschrieben (Groß beginnen, mit Punkt beenden). Es sollte nicht beginnen mit "Diese Methode macht ..." oder "Diese Klasse ist ...". Ein gutes Beispiel wäre "Berechnet die Steuerrückerstattung."
  - Danach kommen die Details, die in der generierten Dokumentation erst durch Aufklappen der Elemente sichtbar sind. Erklären Sie, wieso der Code was machen soll und welche Designentscheidungen getroffen wurden (und warum).
- Leerzeilen gliedern den Text in Absätze. Neue Absätze werden mit einem `<p>` eingeleitet. (Ausnahmen: Wenn der Text mit `<ul>` o.ä. beginnt oder der Absatz mit den Block-Tags.)
- Die "Block-Tags" `@param`, `@return`, `@throws`, `@deprecated` werden durch einen Absatz von der restlichen Beschreibung getrennt und tauchen in exakt dieser Reihenfolge auf. Die Beschreibung dieser Tags ist nicht leer - anderenfalls lässt man das Tag weg. Falls die Zeile für die Beschreibung nicht reicht, wird umgebrochen und die Folgezeile mit vier Leerzeichen (beginnend mit dem `@`) eingerückt.
  - Mit `@param` erklären Sie die Bedeutung eines Parameters (von links nach rechts) einer Methode. Beispiel: `@param date Tag, Wert zw. 1 .. 31`. Wiederholen Sie dies für jeden Parameter.
  - Mit `@return` beschreiben Sie den Rückgabetyp/-wert. Beispiel: `@return Anzahl der Sekunden seit 1.1.1970`. Bei Rückgabe von **`void`** wird diese Beschreibung weggelassen (die Beschreibung wäre dann ja leer).
  - Mit `@throws` geben Sie an, welche "checked" Exceptions die Methode wirft.
  - Mit `@deprecated` können Sie im Kommentar sagen, dass ein Element veraltet ist und möglicherweise mit der nächsten Version o.ä. entfernt wird. (siehe nächste Folie)

=> Dies sind die Basis-Regeln aus dem populären Google-Java-Style (Google Open Source 2022).

## 3 Veraltete Elemente

```
/**
 * Beschreibung Beschreibung Beschreibung.
 *
```

```
4   * @deprecated As of v102, replaced by <code>Foo.fluppie()</code>.
5   */
6  @Deprecated
7  public void wuppie() {}
```

- Annotation zum Markieren als “veraltet” (in der generierten Dokumentation): `@deprecated`
- Für Sichtbarkeit zur Laufzeit bzw. im Tooling/IDE: normale Code-Annotation `@Deprecated`

Dies ist ein guter Weg, um Elemente einer öffentlichen API als “veraltet” zu kennzeichnen. Üblicherweise wird diese Kennzeichnung für einige wenige Releases beibehalten und danach das veraltete Element aus der API entfernt.

# 4 Autoren, Versionen, ...

```
1  /**
2   * Beschreibung Beschreibung Beschreibung.
3   *
4   * @author  Dagobert Duck
5   * @version V1
6   * @since   schon immer
7   */
```

- Annotationen für Autoren und Version: `@author`, `@version`, `@since`

Diese Annotationen finden Sie vor allem in Kommentaren zu Packages oder Klassen.

# 5 Was muss kommentiert werden?

- Alle **`public`** Klassen
- Alle **`public`** und **`protected`** Elemente der Klassen

- Ausnahme: `@Override` (An diesen Methoden *kann*, aber *muss* nicht kommentiert werden.)

Alle anderen Elemente bei Bedarf mit *normalen* Kommentaren versehen.

## 5.1 Beispiel aus dem JDK: ArrayList

Schauen Sie sich gern mal Klassen aus der Java-API an, beispielsweise eine `java.util.ArrayList`:

- Generierte Dokumentation: zu “ArrayList” runterscrollen bzw. direkt
- Quellcode: ArrayList.java

### 5.2 Best Practices: Was beschreibe ich eigentlich?

Unter Documentation Best Practices finden Sie eine sehr gute Beschreibung, was das Ziel der Dokumentation sein sollte. Versuchen Sie, dieses zu erreichen!

## 6 Wrap-Up

- Javadoc-Kommentare sind normale Block-Kommentare beginnend mit `/**`
- Generierung der HTML-Dokumentation mit `javadoc *.java`
- Erste Zeile ist eine Zusammenfassung (fast immer sichtbar)
- Längerer Text danach als "Description" einer Methode/Klasse
- Annotationen für besondere Elemente: `@param`, `@return`, `@throws`, `@deprecated`

- Faustregel: Alle **`public`** und **`protected`** Elemente mit Javadoc kommentieren!

## 7 Zum Nachlesen

- Ullenboom (2021, Kap. 23.4)
- Google Open Source (2022, Kap. 7)

---

**Lernziele**

- k2: Ziel der Javadoc-Dokumentation verstehen
- k2: Typischen Aufgabe von Javadoc-Kommentaren verstehen
- k3: Dokumentation öffentlich sichtbarer Elemente mit Javadoc
- k3: Schreiben einer sinnvollen Summary
- k3: Einsatz von Annotationen zur Dokumentation von Parametern, Rückgabetypen, Exceptions, veralteten Elementen

**Challenges**

Betrachten Sie die Javadoc einiger Klassen im Dungeon-Projekt: dojo.rooms.LevelRoom, dojo.rooms.MonsterRoom, und contrib.components.HealthComponent.
Stellen Sie sich vor, Sie müssten diese Klassen in einer Übungsaufgabe nutzen (das könnte tatsächlich passieren!) ...
Können Sie anhand der Javadoc verstehen, wozu die drei Klassen dienen und wie Sie diese Klassen benutzen sollten?
Vergleichen Sie die Qualität der Dokumentation. Was würden Sie gern in der Dokumentation finden? Was würden Sie ändern?

---

**Quellen**

Google Open Source. 2022. „Google Java Style Guide“. 2022. https://google.github.io/styleguide/javaguide.html.
Ullenboom, C. 2021. *Java ist auch eine Insel*. 16. Aufl. Rheinwerk-Verlag. https://openbook.rheinwerk-verlag.de/javainsel/index.html.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 09509bb (tooling: shift headings (use h1 as top-level headings) (#993), 2025-04-23)